

Lecture 3: ILP & Hazards

What is instruction Level Parallelism (ILP) ?

❑ ILP is a technique where instructions can execute partially or fully in parallel

❖ Pipelining (partially)

❖ Multiple issue (Fully)

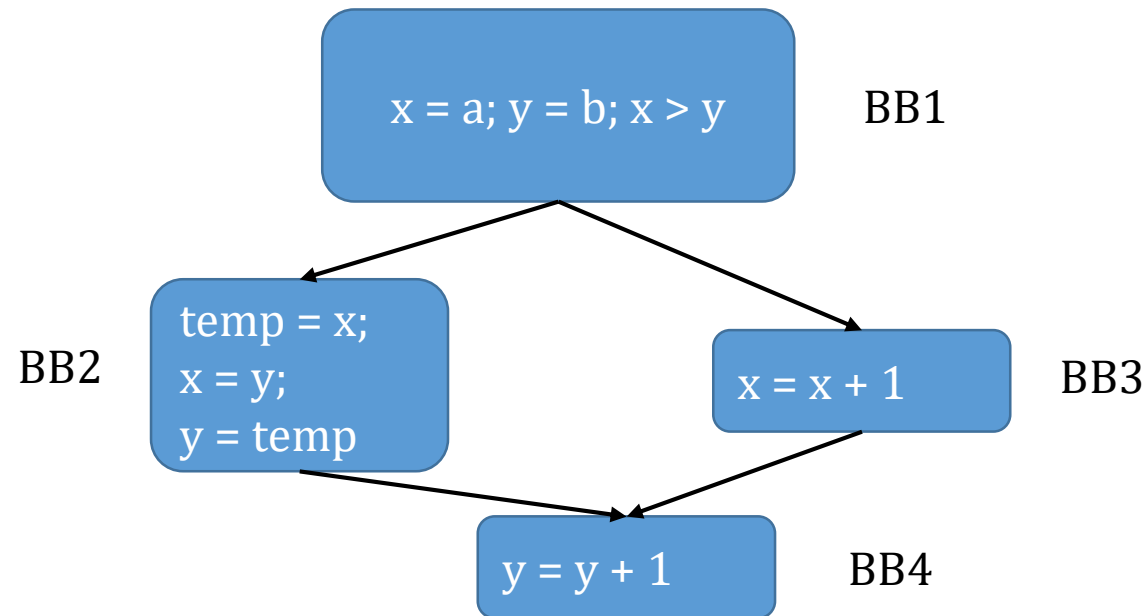
IF	ID	EX	M	WB		
	IF	ID	EX	M	WB	
		IF	ID	EX	M	WB

IF	ID	EX	M	WB		
IF	ID	EX	M	WB		
	IF	ID	EX	M	WB	
	IF	ID	EX	M	WB	
		IF	ID	EX	M	WB
		IF	ID	EX	M	WB

Where to look for ILP?

- ❑ In Basic Block: Code sequence with no branches in or out excepts at entry and exit

```
x = a;  
y = b;  
if (x>y) {  
    temp = x;  
    x = y;  
    y = temp;  
}  
else  
    x = x+1;  
y = y+1;
```



- Dynamic Branch Frequency is 15-25%
 - Usually 4-6 instructions between branches
- Instructions in BB are often dependent
 - Must exploit ILP across BBs

Where to look for ILP?

- Loop level parallelism: Loop iterations are independent and can be executed in parallel

```
for (i=0; i<1000; i++)  
    a[i] = b[i] + c[i];
```

- ILP cores turn LLP into ILP using
 - Dynamic loop unrolling using branch prediction
 - Speculative execution of predicted instruction
 - Register renaming to avoid name dependency
 - ...

Program Order and ILP

❑ In principle HW/SW must preserve program order:

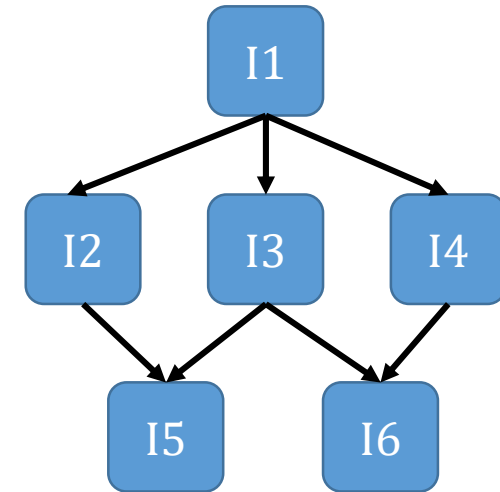
- ❖ Order in which instructions are executed
 - If they are executed sequentially one at a time
 - As determined by the original program

❑ Technique going to exploit: preserve program order so that program outcome does not change

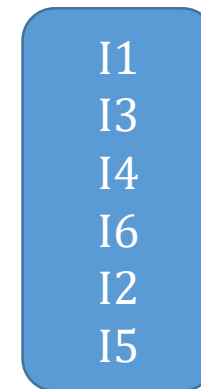
Program Order



data dependency graph



Legal execution orders



non-legal



Dependences

❑ True dependences – Data Dependences

- ❖ Cannot be re-ordered and needs to be executed in order

❑ Name Dependences – no flow of data b/n the instructions, dependence cause by the same name (register)

- ❖ Anti-Dependence

- ❖ Output Dependence

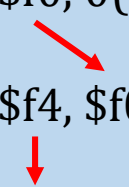
❑ Control Dependences

Data Dependences – RAW Hazard

- ❑ Instruction K is data dependent on inst I if
 - ❖ I produces result used by K , or
 - ❖ K data dependent on J and J data dependent on I

- ❑ Data dependence may result in RAW hazard
 - ❖ A dependence result in a hazard only if stall cycle is required

I :	l.s	\$f0, 0(\$f1)
J :	add.s	\$f4, \$f0, \$f2
K :	s.s	\$f4, 0(\$f1)



I	IF	ID	EX	M	WB				
J		IF	ID	X	A1	A2	A3	A4	
K			IF	X	EX	X	X	X	M

Name Dependences - Anti-Dependence

❑ Anti-Dependences occurs if I reads variable written by subsequent J

❖ WAR Hazard

❑ In principle cannot re-order I and J

❑ Called name dependence because I and J use same name (reg and mem location) but no flow of data

❖ can remove dependence by renaming variable

$K:$	$B = 3$
$I:$	$A = B + 1$
$J:$	$B = 7$

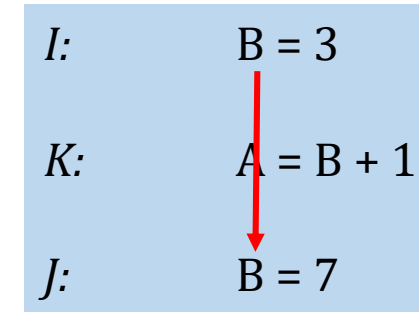


$K:$	$B = 3$
$I:$	$A = B + 1$
$J:$	$B1 = 7$

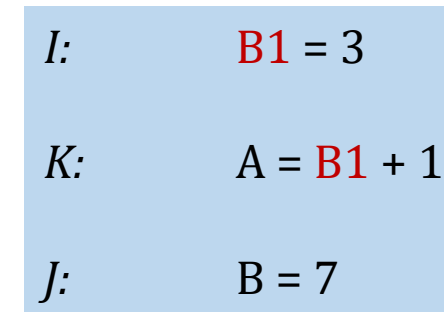
Name Dependences- Output Dependence

❑ Output dependence occurs if I and J write same variable

❖ WAW hazard



Also **name dependence** – can be removed by **renaming**

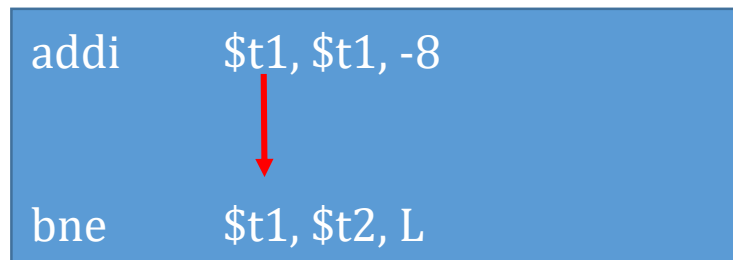


Read after Read constitute no dependence or hazards

Dependence vs Hazards

- ❑ Dependences are program properties
- ❑ Hazards are properties of micro-architecture

- Data Dependence results in RAW hazards
 - **If** branch is executed in ID stage
 - **Not if** branch is executed in EX stage



<i>addi</i>	IF	ID	EX	M	WB		
<i>bne</i>		IF	X	ID	EX	M	WB

<i>addi</i>	IF	ID	EX	M	WB	
<i>bne</i>		IF	ID	EX	M	WB

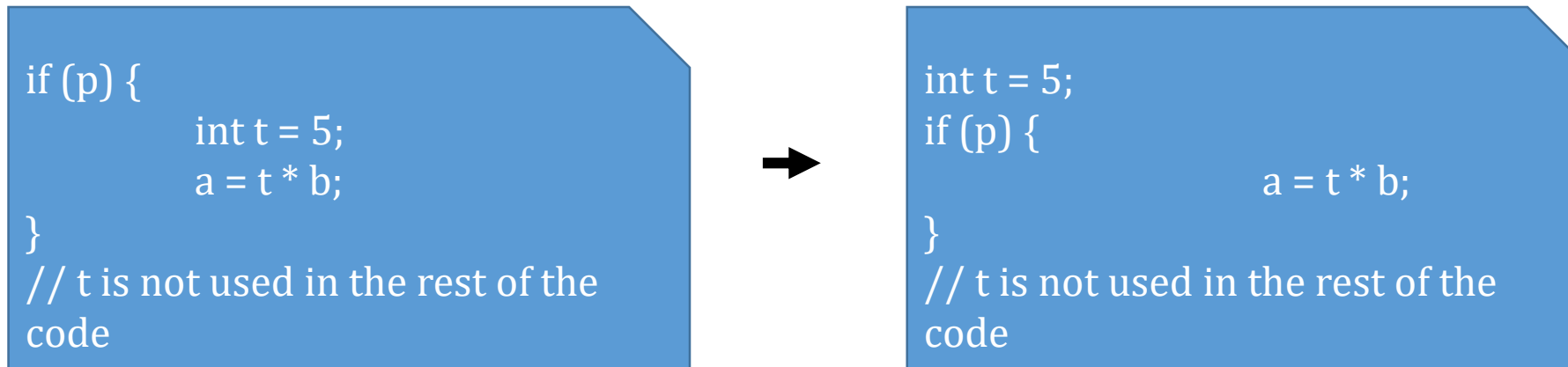
Control Dependences

if (p1)	S1;	S1 control dependent on p1
else	S2;	S2 control dependent on p1
if (p2)	S3;	S3 control dependent on p2

- In principle
 - Control dependent instruction **cannot** be moved before branch
 - Instruction that is not control dependent on branch **cannot** be made control dependent

Scheduling Instructions across branches

- Control dependent instruction can be moved before branch if its result used in only one path



- Similar: Instruction that is not control dependent on a branch **can** be made control dependent if its results are used only in that path

Control Dependences

- Control dependences $\neq$ fundamental performance limits
 - Violations allow only when correctness is preserved
 - May execute instructions that do not have to execute provided they can be undone
 - **speculative execution**
- Properties critical to program correctness are
 - dataflow
 - exception behaviour

Exception Behaviour

- Preserve exception behaviour: instruction reordering may not cause exception that would not occur in program order

```
if ( p != NULL ) {  
    d = * p;  
    q = d * d;  
}
```

```
beqz    $t1, endif  
l.s     $f0, 0($t1)  
mul.s   $f1, $f0, $f0  
endif:
```

```
l.s     $f0, 0($t1)  
beqz    $t1, endif  
mul.s   $f1, $f0, $f0  
endif:
```

- May cause memory protection violation that should not have occurred
- Want to ignore exception if branch is taken

Basic idea of Dynamic Scheduling

In-order Execution													
div.d	F0, F1, F2	IF	ID	D1	D2		...	D20					
add.d	F3, F0, F4		IF	ID	X	X	X	X	A1	A2	A3	A4	
sub.d	F12, F10, F11			IF	X	X	X	X	ID	X	X	X	A1

Assume divide takes 20 cycle

- Since there is a RAW hazards between the div and add the add has to wait for the divide to complete
- The sub also need to stall in its fetch state. Though it has no dependences
- The basic idea of dynamic scheduling is to allow instruction to execute out of order
- It can also be observed that sub instruction further stall due to one floating point adder

Dynamic Scheduling

In-order Execution													
div.d	F0, F1, F2	IF	ID	D1	D2		...	D20					
add.d	F3, F0, F4		IF	ID	X	X	X	X	A1	A2	A3	A4	
sub.d	F12, F10, F11			IF	X	X	X	X	ID	X	X	X	A1

Out-of-Order (OoO) Execution													
div.d	F0, F1, F2	IF	ID	D1	D2		...		D20				
add.d	F3, F0, F4		IF	ID	X	X	X	X	X	A1	A2	A3	A4
sub.d	F12, F10, F11			IF	ID	A1	A2	A3	A4				

Analogy of Dynamic Scheduling

❑ One Lane Road

- ❖ If one car stops, all cars behind it stops

❑ Multiple Lanes

- ❖ If one cars stops, cars behind it can take over

Tomasulo's Algorithm

- ❑ Implemented in IBM 360/961 even before caches were invented
- ❑ Goal: High performance without special compilers
- ❑ Limitations of the IBM 360/961 processors
 - ❖ Only 4 FP registers which limits the effectiveness of compiler scheduling
 - ❖ This motivated Tomasulo to get more effective registers -> renaming in hardware
 - ❖ Further the IBM360/961 has long memory access and FP delays which Tomasulo algorithm is designed to overcome
- ❑ Tomasulo's algorithm popularity since 1960 and in prevalent these days too- Descendent like Alpha 21264, HP 8000. Pentium 4, Power8, Core i5, i7, ...

Robert Tomasulo



Award
1997 Eckert-Mauchly Award
"For the ingenious Tomasulo's algorithm, which enabled out-of-order execution processors to be implemented."

Key hardware Structures used in Tomasulo Algorithm

div.d F0, F1, F2
add.d F3, F0, F4
sub.d F12, F10, F11

- To allow **sub.d** to proceed and buffer **add.d** somewhere
 - In Tomasulo algorithm the buffers are term as **Reservation Stations (RSs)**
- To allow **add.d** to proceed when its operand becomes available, RSs must be informed when result available
 - In Tomasulo algorithm results broadcasted on **Common Data Bus (CDB)**

Tomasulo Pipeline Phases

□ IF

- ❖ Fetch next instruction into a FIFO queue of pending instructions

□ Issue

- ❖ Get next instruction from head of instruction queue
- ❖ If matching RS free (no structural hazard), issue instruction into RS
 - with operands values if they are currently in registers
 - Otherwise, with identifiers of RSs that will produce operands

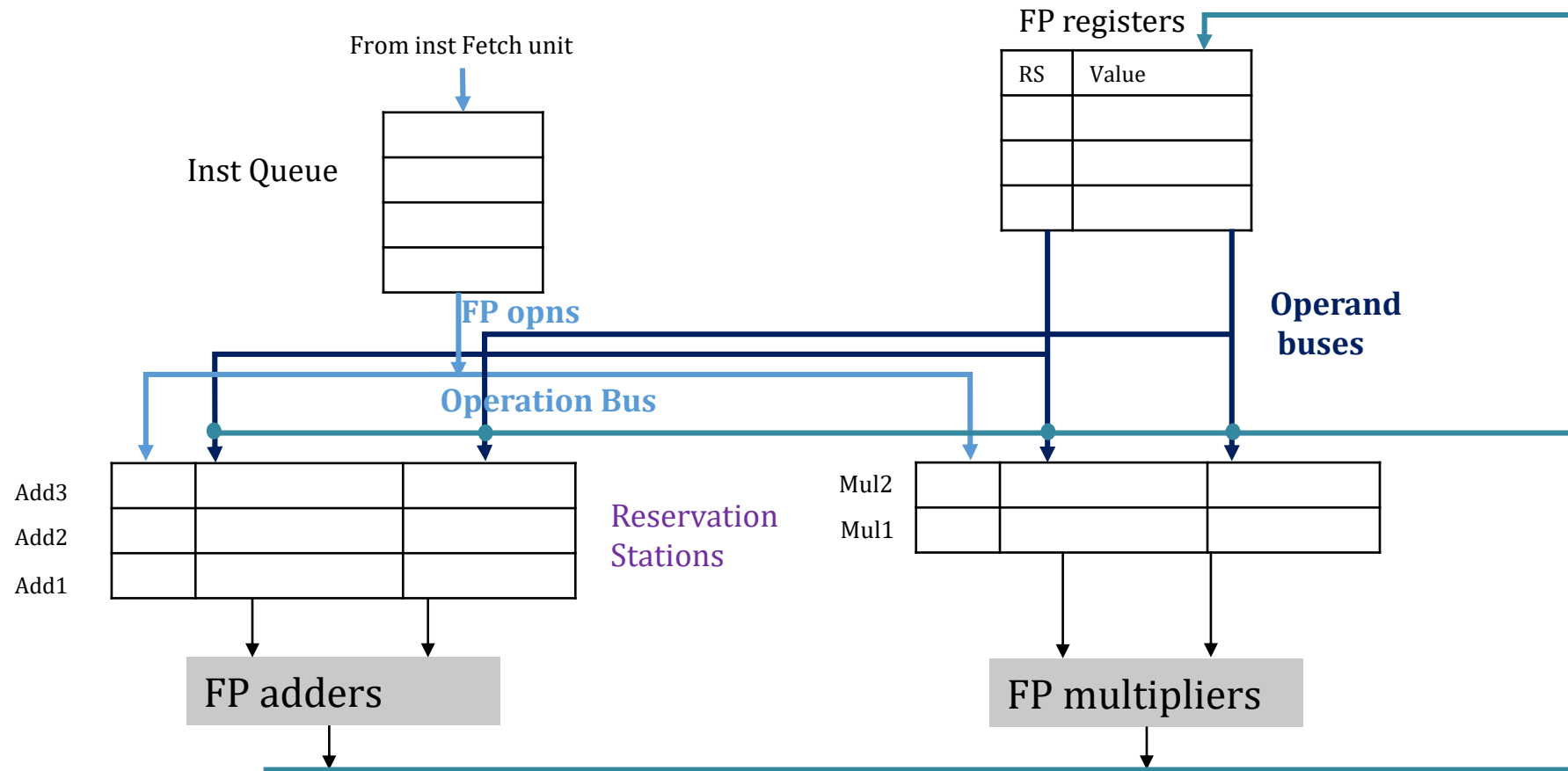
□ Execute

- ❖ Execute when all operands available (no RAW) and FU free
- ❖ If not, monitor the CDB for result

□ Write Result

- ❖ Write result on CDB to all awaiting RSs and register file
- ❖ Mark RS free

Tomasulo- Basic EPU Structure



RS Structure

❑ Each **RS** has 7 fields

- ❖ **op**: operation to perform
- ❖ **RS1**: RS that will produce 1st operand (0: operand available)
- ❖ **RS2**: RS that will produce 2nd operand
- ❖ **Val1**: value of 1st operand
- ❖ **Val2**: Value of 2nd operand
- ❖ **Imm/addr**: holds immediate or effective address
- ❖ **Busy**: RS occupied

Example

- Add
- 1st operand being produced by RS Mul2
- 2nd operand available in register

op	RS1	RS2	Val1	Val2	Imm/add	Busy
add	Mul2	0	n/a	12.55	n/a	1

Register Structure

❑ Each register has field RS to track whether the register has the value in register or awaiting from RS.

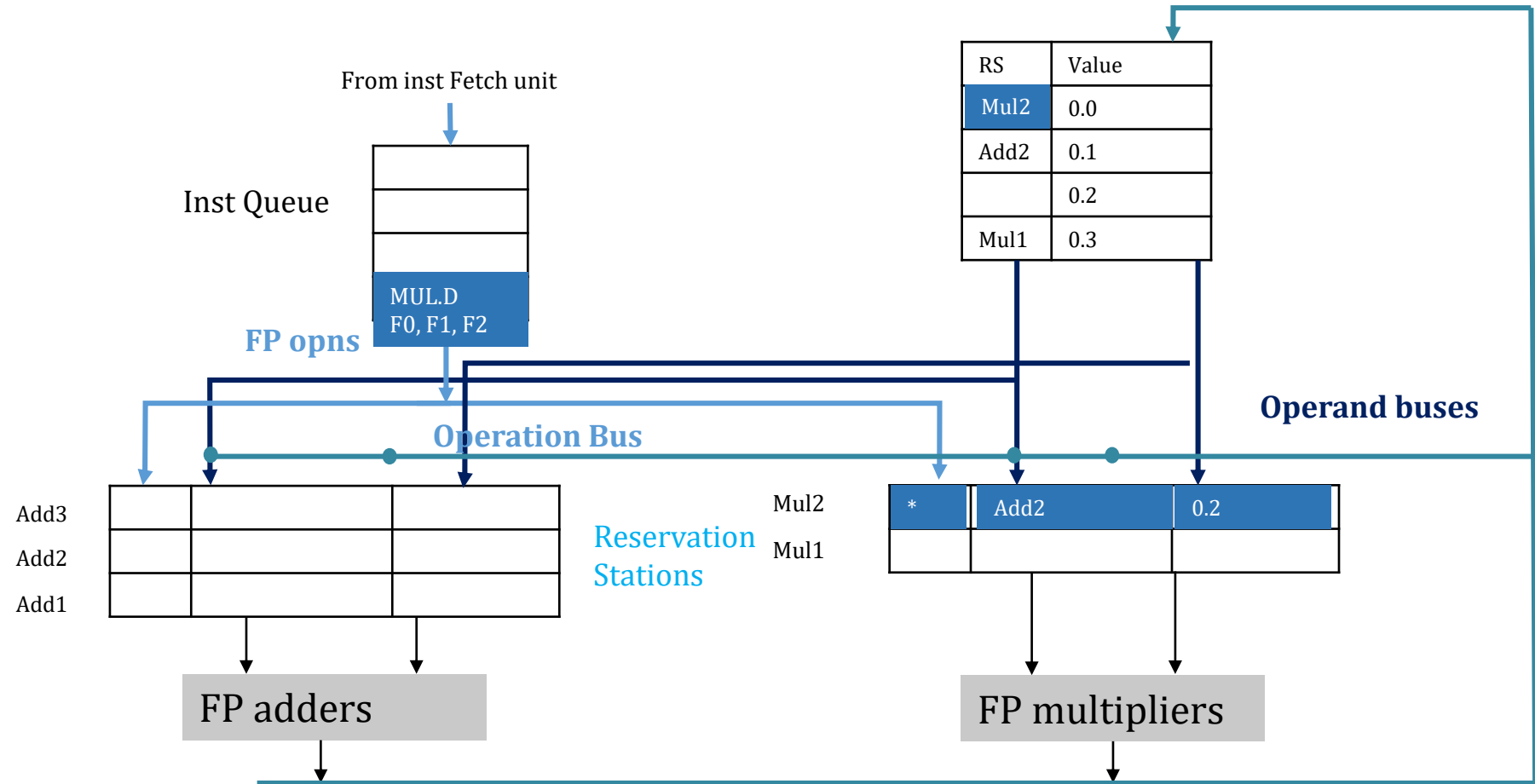
- ❖ RS ID that will produce this value
- ❖ Blank/ 0 if not applicable

RS	Value
Mul1	
Add1	

1st and 3rd registers RS fields are blank -> valid data
2nd and 4th are awaiting from RSs (Mul1 and Add1 to complete)

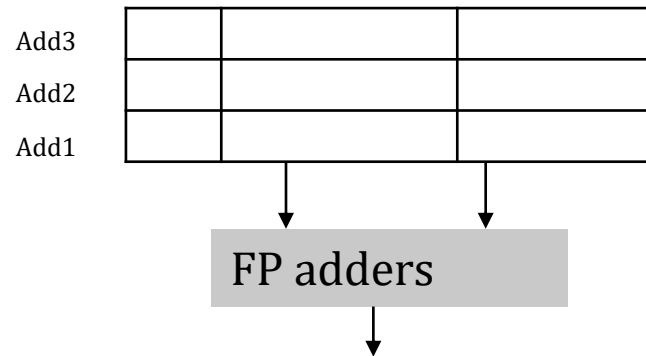
Issue FP Instruction

- ❑ if RS available
- ❑ if src operands
 - ❖ in registers with values
 - ❖ “in-flight”: link with producing RS



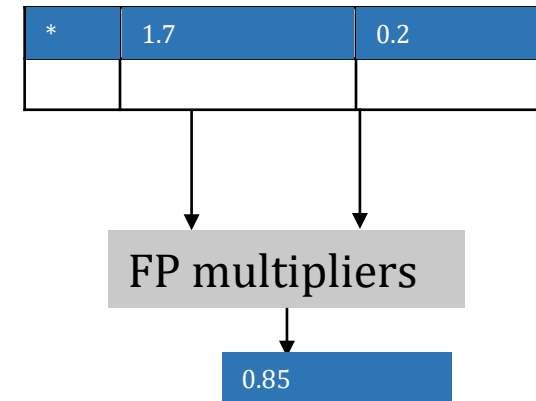
Execute FP Instn

- ❑ Execute when all operands available (no RAW)
- ❑ Several instn may become ready at same time
 - ❖ For FP RSs, choice is arbitrary, FIFO is usual
 - ❖ Ld/st execute in order (will be discussed)



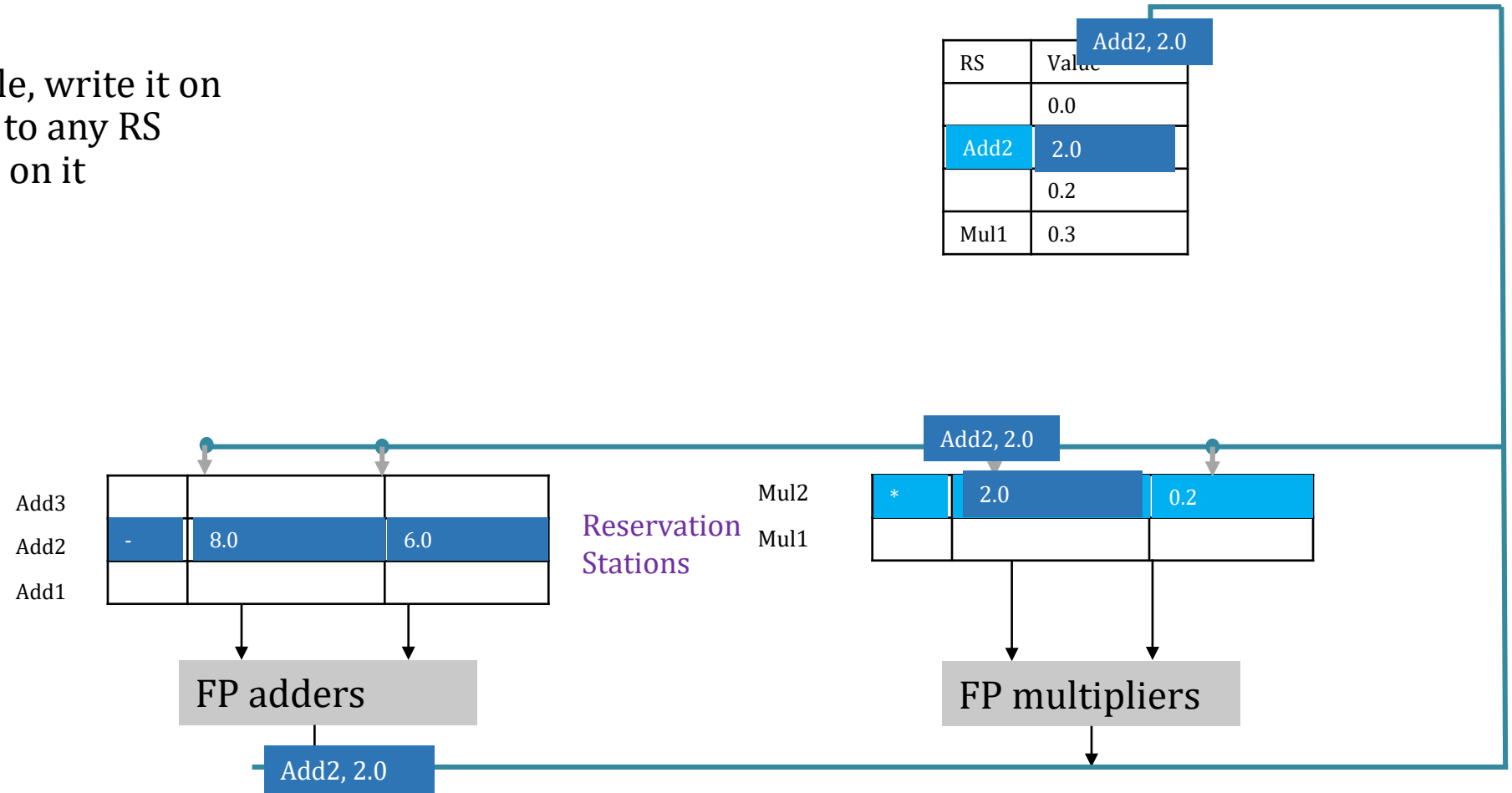
Reservation
Stations

Mul2
Mul1



Write Result

- ❑ When result available, write it on CDB and from there to any RS and register waiting on it



Thank you